PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Table of Contents

1. Project Overview 2
2. Background & Business Objective 2
3. Product Purpose 3
4. System Functionalities 4
5. User Stories & Use Cases 7
6. Features Included (Scope Definition) 8
7. Features Not Included (Scope Control) 8
8. Assumptions & Constraints 8
9. Risks & Questions Throughout Development 9

© UMHackathon 2026

Project Name: Hospital AI Decision Intelligence System (HaiDIS)

Version: 1.0

1. Project Overview

1.1 Problem Statement

Malaysia's public healthcare system is facing increasing pressure due to overcrowded hospitals, staff shortages, and inefficient resource allocation. These challenges result in long patient waiting times, overworked healthcare staff, rising operational costs, and inconsistent quality of care. Current hospital management approaches in Malaysia lack real-time insights and data-driven decision-making tools, making it difficult to balance cost, efficiency and patient outcomes.

1.2 Target Domain

Our solution focused on hospital operations optimization, real-time monitoring, and data-driven decision support to improve resource allocation and service efficiency.

1.3 Proposed Solution Summary

Hospital AI Decision Intelligence System (HaiDIS) acts as a platform that helps administrators make smarter, data-driven decisions in real time. It provides live hospital metrics (patients, staff, beds, average waiting time), AI-generated recommendations, cost vs efficiency trade-off analysis, and an interactive What-If Simulator to test different scenarios. By transforming complex hospital data into actionable insights, the system enables better resource planning, reduces inefficiencies, and improves overall healthcare delivery.

2. Background & Business Objective

2.1 Background of the Problem

For years, hospital management in Malaysia has relied on manual planning and static reporting systems to allocate resources such as staff, beds, and equipment. Decisions are often based on historical data rather than real-time conditions, making it difficult to respond quickly to sudden patient surges or staff shortages. This leads to inefficient workflows, overcrowding, underutilisation or overuse of resources, and increased pressure on healthcare workers. The lack of integrated, intelligent systems limits the ability of hospital administrators to make fast, informed, and data-driven decisions.

© UMHackathon 2026

2.2 Importance of Solving This Issue

Improving hospital resource management can significantly reduce patient waiting times, optimise staff workload, and enhance the overall quality of care. By enabling real-time monitoring and predictive decision-making, hospitals can respond more effectively to changing conditions, reduce operational costs, and prevent burnout among healthcare workers. Solving this issue also contributes to a more sustainable healthcare system, where resources are used efficiently while maintaining high standards of patient care.

2.3 Strategic Fit / Impact

This solution aligns with the growing adoption of AI and digital transformation in healthcare, supporting Malaysia's vision for smart hospitals and data-driven public services. By integrating AI-powered recommendations, real-time analytics, and simulation tools, the system enhances operational efficiency and decision-making capabilities.

3. Product Purpose

3.1. Main Goal of the System

- To develop an AI-powered decision intelligence system for hospital management.
- To optimise the allocation of hospital resources such as staff, beds, and facilities.
- To improve overall operational efficiency within healthcare settings.
- To reduce patient waiting times and enhance service delivery.
- To provide real-time data insights for better monitoring and responsiveness.
- To enable predictive analysis for more accurate and proactive decision-making.

3.2. Intended Users (Target Audience)

- **Hospital Administrators or Management**
  To oversee operations and make strategic decisions.
- **Healthcare Operations Managers**
  To manage staff, beds, and daily workflow efficiently.
- **Doctors & Nurses (Supervisory Roles)**
  To monitor workload and resource availability.
- **Hospital IT & Data Teams**
  To manage system integration and data monitoring.
- **Government Health Authorities / Policy Makers**
  To analyse trends and improve public healthcare planning.

© UMHackathon 2026

4. System Functionalities

4.1. Description

The system operates as an AI-powered decision intelligence system for hospital resource management. It analyzes data such as number of patients, waiting time, staff availability, and bed capacity to generate suitable recommendations to improve healthcare service. The system implements a predictive simulation model that illustrates the impacts of actions made, aiming to assist hospital administrators in making decisions that can optimize operations, reduce waiting time, and improve service efficiency. All recommendations and data are shown through an interactive and clear dashboard interface for easy access and visualization.

4.2. Key Functionalities

- **AI Recommendations:** The system analyzes hospital data to generate actionable recommendations such as reallocating staff and resources, and adjusting shifts. The recommendations are to assist administrators in making decisions that can enhance services and resource allocation.
- **Cost vs Efficiency Analysis:** The system evaluates the trade-offs between operational cost and service efficiency for each generated recommendation. It estimates the percentage of changes in cost for adding staff or resources and how much average waiting time is reduced by doing so. Each analysis is provided with clear explanations and insights for better user understanding.
- **What-If Simulation:** The system allows users to simulate changes in hospital resources such as adding bed or staff, and predicts the impact of each action on waiting time, overall operational costs and number of patients served per day, providing clear visualization for improved decision making.

4.3. AI Model & Prompt Design

4.3.1. Model Selection:

HaiDIS uses ILMU GLM-5.1 provided by Z.AI, the collaborating partner for this project. As the designated language model for this hackathon, GLM-5.1 was integrated as the core AI engine powering HaiDIS's recommendation system. Beyond the collaborative context, GLM-5.1 proved technically well-suited for the product's requirements for the following reasons:

Structured JSON enforcement. GLM-5.1 supports response_format: json_object, which forces the model to return valid JSON only. This is critical for HaiDIS since the frontend directly renders the parsed response — any free-form text output would break the dashboard.

© UMHackathon 2026

Instruction-following reliability. The model consistently adheres to constrained system prompts, respecting field constraints such as title length limits, exact suggestion count, and priority values of High, Medium, or Low.

OpenAI-compatible API. GLM-5.1 exposes an OpenAI-compatible interface, allowing HaiDIS to integrate using the standard OpenAI Python client with minimal overhead, reducing development time and keeping the codebase maintainable.

4.3.2. Prompting Strategy:

HaiDIS uses two different prompting strategies across its two AI-powered features:

- Feature 1 - Dashboard AI Recommendations: Zero-shot prompting.
  The system prompt precisely defines the output schema, rules, and constraints, while the user message supplies the current hospital simulation data as a structured JSON payload. No worked examples are provided. This fits the dashboard feature because the input is always a consistent, structured JSON object, thus the model does not need examples to understand the task.

- Feature 2 - What-If AI Recommendation: Zero-shot prompting with plain text output.
  The prompt instructs GLM to respond in exactly three plain text sections: Analysis, Recommendation, and Cost Impact without JSON or markdown. The prompt supplies the simulation result directly (wait time, patients served, doctor load, nurse load, added cost, status). This fits the what-if feature because the output is narrative rather than structured, and the prompt is self-contained enough that no examples are needed.

  Although the underlying data updates continuously as management staff enter new records, the payload that reaches the model is always a consistent, structured object so the model does not need examples to understand either task. Multi-step agentic prompting was ruled out as both tasks are self-contained within a single inference call.

4.3.3. Context & Input Handling:

HaiDIS receives its operational data from the hospital management unit's database, which is maintained and updated by management staff outside of the HaiDIS system. Each time new data is entered by staff, the database reflects the latest hospital metrics. HaiDIS reads from this source to ensure recommendations are always based on current conditions.

© UMHackathon 2026

Before any model call is made, analytics.py queries and processes the latest available data, distilling it into a compact JSON summary via build_prompt_payload(). The context window contains the system prompt and this aggregated payload, which includes zone status, wait times, bed and staff utilisation, peak specialization load, and pre-computed rule-based suggestions.

Because the payload is always a small, consistently structured JSON object regardless of how much data exists in the underlying database, no truncation or chunking is required on HaiDIS's end. The max_tokens parameter is capped at 2500, enforced at the API call level inside ai_service.py. HaiDIS does not expose any free-text input interface to end users, so there is no risk of oversized or unstructured inputs reaching the model.

4.3.4. Fallback & Failure Behavior:

HaiDIS implements different fallback strategies for each AI feature:

For the Dashboard recommendations fallback, if the GLM API key is missing or the API call fails for any reason, ai_service.py catches the exception and returns pre-computed rule-based suggestions from analytics.py. These are zone-aware (Red, Yellow, and Green Zone) each produce different deterministic recommendations so the dashboard always displays something meaningful. Additionally, a simulation-based fallback (dashboard_fallback_recommendations()) is available if the analytics pipeline itself fails.

As for the What-if AI recommendation fallback, if the GLM call in ask_ai_for_simulation() fails, the system returns an HTTP 500 error with the error message and the raw simulation result. Unlike the dashboard, there is no pre-computed fallback for the what-if AI recommendation. The simulation result itself (wait time, patients served, staffing cost, status) is still returned so the user has the quantitative output even without the AI narrative.

Hallucination mitigation. For the dashboard feature, the system prompt explicitly instructs GLM to base responses only on the provided data, and the parsed response is validated for a non-empty suggestions list before being accepted. For the what-if feature, the response is parsed line-by-line for the three expected sections. If any section is empty or missing, a ValueError is raised. Neither feature implements semantic validation for contextually incorrect content within a structurally valid response.

There is no retry mechanism or human escalation path in either feature in the current version.

5. User Stories & Use Cases

- **User Stories:**
  - As a hospital administrator, I want to view current hospital resource data so that I can understand the overall operational situation.
  - As a hospital administrator, I want to receive AI-generated recommendations so that I can make faster and more informed decisions.
  - As a hospital administrator, I want to simulate changes in resources (e.g., adding staff or beds) so that I can evaluate their impact before making decisions.

- **Use Cases (Main Interactions):**
  - **AI recommendations Flow:** The user clicks on the 'Generate' button. The system processes the input data using predefined logic and AI models to analyze the situation. It will then generate appropriate actionable recommendations for the current situation, and display the recommendations on the dashboard.
  - **Simulation and Prediction Flow:** The user enters the number of variables such as beds, doctors and nurses. The system recalculates and predicts the impact of these changes on key metrics such as waiting time and workload.

- **Use Case Diagram:**

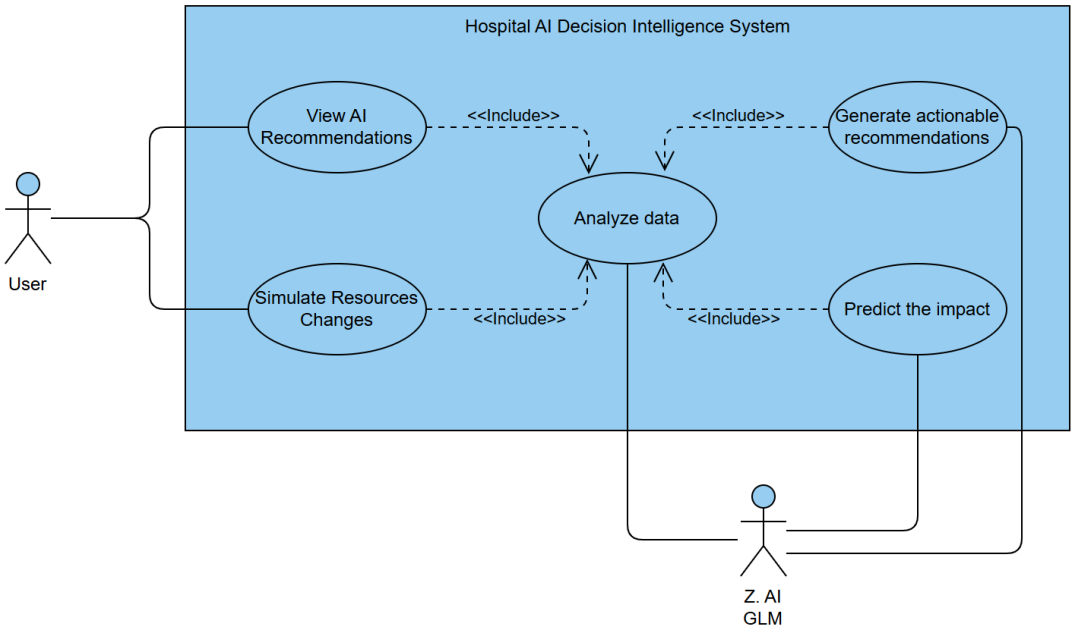

Figure 5.1: Use Case Diagram

6. Features Included (Scope Definition)

- **AI-powered Decision Recommendation:** The system generates recommendations using predefined logic and simulated AI reasoning based on input data.
- **Interactive Dashboard:** The system implements an interactive and visually engaging interface to display clear visualization of hospital information.
- **What-If Simulation Feature:** The simulation model is used for predicting impact of resource modifications to assist in decision making.
- **Cost vs Efficiency Analysis:** The system shows trade-offs between operational cost and efficiency for better understanding of the purpose behind recommendations given.
- **Automated Data Generation for Hospital Resources:** The system relies on automatically generated data for input such as number of patients, staff and beds as the basis for analysis and simulation.

7. Features Not Included (Scope Control)

- **Real-time Hospital Data Integration:** The system will not connect to live hospital databases or Electronic Health Records (EHR). It relies solely on manual user (hospital management staff) inputs for analysis.
- **Live Shift Scheduling & Staff Communication:** While the system provides general shift planning suggestions, it does not include a functional module for staff to clock in/out or receive automated assignments.
- **Predictive Alert System:** Automated "Smart Alerts" for future overstock or low demand risks are not included in this version.
- **User Authentication & Multi-Role Access:** The current scope does not support secure login, user profiles, or different access levels (e.g., separate views for doctors vs. administrators).
- **Financial Transaction Processing:** The "Cost vs Efficiency" feature is for estimation and simulation purposes only; it does not handle actual hospital billing or payment processing.
- **Mobile Application:** This project is strictly a web-based platform and does not include a native mobile app (iOS/Android).

8. Assumptions & Constraints

- **LLM Cost Constraint:**
  - **Estimation:** Based on the current architecture , an average user session (involving inputting data, running 1-2 "What-If" simulations, and getting AI recommendations) is estimated to cost between 1,000 to 3,000 tokens.

© UMHackathon 2026

- **Design Decision for Scalability:** To keep inference costs manageable, we will implement Prompt Engineering with Structured Output (JSON). By forcing the AI to return only specific numerical values and short strings rather than long conversational prose, we significantly reduce the output token count. Additionally, common "What-If" scenarios (like adding +1 doctor) can be cached so the system doesn't call the API for identical repeated simulations.

- **Technical Constraints:**
  - **Architecture Limitation:** The backend is strictly built using **Python**, and the frontend is limited to **HTML, CSS, and JavaScript**.
  - **Deployment:** The platform is optimized for web deployment via **Vercel**; native mobile support (Swift/Java) or complex frontend frameworks like React are not supported in this version.
  - **API Dependency:** The system's "intelligence" is entirely dependent on the availability and response time of the **GLM API**. If the API is down, the recommendation and simulator features will not function.
- **Performance Constraints:**
  - **Recommendation Latency:** High-quality AI resource recommendations and "What-If" analysis may take **3 to 5 seconds** to generate due to API processing times.
  - **Input Limits:** To prevent system overload and high costs, the **Hospital Resource Input** will have character limits, and the **What-If Simulator** will have defined ranges (e.g., users cannot input 1,000 extra doctors) to ensure the AI remains within logical bounds.
- **User Input:**
  - **Human-in-the-Loop:** The system is a **Decision Intelligence System**, meaning it is designed to assist, not replace, human managers. The AI-generated resource allocations and shift plans require human approval before implementation.
  - **Prompt Dependency:** The system cannot generate insights without specific data provided in the **Hospital Resource Input** section (Patients, Doctors, Nurses, and Beds).

9. Risks & Questions Throughout Development

- **Real-Time Data Synchronisation**
  - **Risk:** What happens if the real-time hospital data (e.g. patient count, available beds, staff on duty) is not updated promptly or becomes delayed, leading to inaccurate dashboard insights and AI recommendations?
  - **Mitigation:** The system ensures data reliability by implementing scheduled auto-refresh intervals and backend data synchronization mechanisms. A fallback caching system is used to temporarily store the most recent valid data in case of delays. Additionally, timestamps are displayed to indicate the last update, ensuring

© UMHackathon 2026

users are aware of data freshness and preventing decisions based on outdated information.

- **Design Similarity:**
  - **Risk:** How are we ensuring that the AI is not generating similar recommendations or layouts across different user scenarios?
  - **Mitigation:** While the dashboard UI remains a fixed "Prototype for design" , the **AI Recommendations (CORE)** are made dynamic through **Contextual Prompting.** By feeding the GLM API specific "Hospital Resource Inputs" —such as the exact number of patients and available beds —the AI is forced to generate unique recommendations tailored to that specific hospital's data, rather than generic advice.
- **What-If Simulation Accuracy:**
  - **Risk:** What happens if the What-If Simulator generates unrealistic or impractical outcomes when users adjust variables such as number of doctors, nurses, or beds?
  - **Mitigation:** The system ensures realistic simulation results by enforcing logical constraints and predefined boundaries on all input variables (e.g. minimum and maximum limits for staff and bed capacity). The simulation model is based on validated assumptions and healthcare operational logic to prevent exaggerated or impossible outputs. If inputs fall outside acceptable ranges, the system will prompt users with warnings or restrict the simulation to maintain accuracy and reliability.
- **Frame Stability:**
  - **Risk:** Does the user need to fix the layout if it is found broken after exporting or during simulation?
  - **Mitigation:** We mitigate this by using a **Responsive Grid System.** The UI is designed to be "Prototype-ready", meaning the layout frames for the "What-If Simulator" and "Simulation Results" are fixed containers that will not break regardless of how many tokens the AI returns.

- **Cost vs Efficiency Calculation Accuracy:**

  - **Risk:** What happens if the system produces incorrect cost versus efficiency calculations, leading to misleading insights and poor decision-making by hospital administrators?

  - **Mitigation:** The system ensures calculation accuracy by implementing validated formulas and cross-check mechanisms for all cost and efficiency metrics. Standardised calculation models are used, and outputs are verified against predefined test datasets. Additionally, sanity checks are applied to detect abnormal values, and clear data inputs are required to minimise calculation errors and maintain reliability.

© UMHackathon 2026

- **Drawing Interpretation**:
  - **Risk:** What will happen if the AI is unable to contextualize the "Flowchart of Operations" or sketches provided during development?
  - **Mitigation:** To avoid logic errors, the system does not rely on the AI to "read" drawings or sketches. Instead, the team will translate the **Architecture and Key Data Flow (DFD)** into structured text/code. If the GLM API receives invalid or nonsensical data that it cannot contextualize, the system will trigger a **"Fallback Error"**, displaying a default message like "Insufficient data for recommendation" to maintain system stability.

© UMHackathon 2026